Load Balancing & Auto Scaling

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

Learning Objectives

- Explain the relationship between regions, availability zones, and fault tolerance
- Compare how AWS, GCP, and Azure organize their global infrastructure
- Define what a load balancer does and why it is essential for cloud-native applications
- Distinguish between Layer 4 and Layer 7 load balancers with concrete examples
- Compare load balancer offerings across AWS, GCP, and Azure
- Explain health checks, sticky sessions, SSL termination, and connection draining
- Define auto scaling and describe how it works with load balancers
- Compare auto scaling mechanisms across AWS, GCP, and Azure
- Choose the appropriate load balancer type for a given application scenario

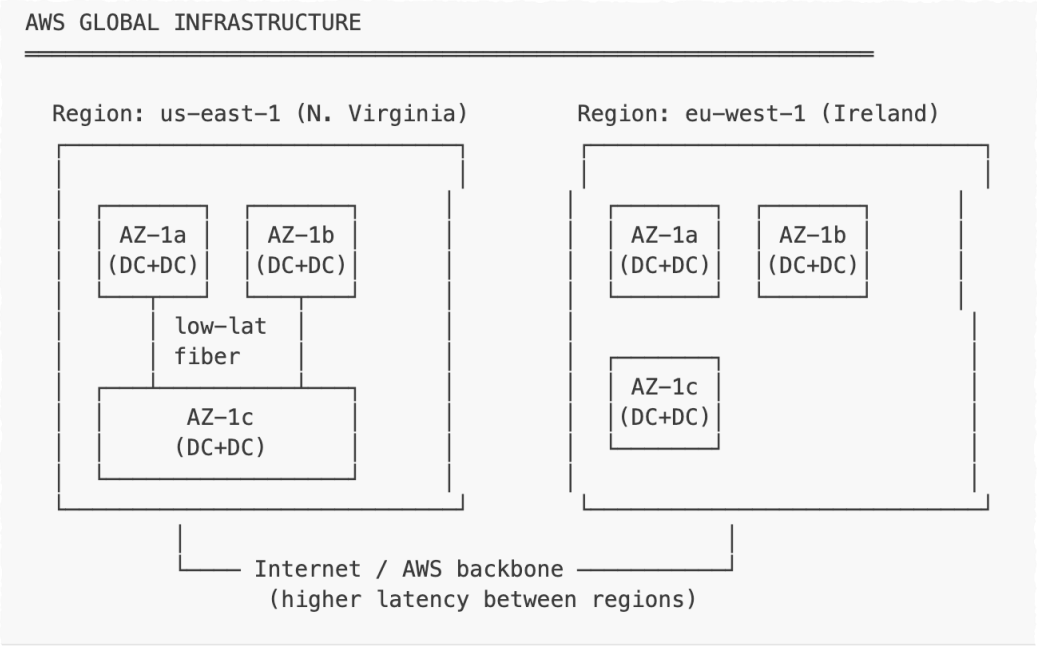
Regions and Availability Zones

Why Geography Matters in Cloud Computing

- When you deploy an application to "the cloud," it runs on physical hardware in a specific location. That location matters for three reasons: latency (how fast users get a response), data residency (legal requirements about where data is stored), and fault tolerance (what happens when hardware or an entire facility fails).
- Cloud providers solve this by organizing their infrastructure into a hierarchy of geographic units. The terminology differs across providers, which is a common source of confusion.

AWS Global Infrastructure

- AWS organizes infrastructure into Regions, Availability Zones, and Edge Locations.
- A **Region** is an independent geographic area (e.g., us-east-1 is Northern Virginia). Each Region is fully isolated from other Regions — no data replicates between Regions unless you explicitly configure it.
- An **Availability Zone (AZ)** is one or more physical data centers within a Region, each with independent power, cooling, and networking. AZs within the same Region are connected by high-bandwidth, low-latency private fiber, but are physically separated (often miles apart) to protect against localized disasters like fires or flooding.
- Key naming conventions: us-east-1a, us-east-1b, etc. — but AZ IDs are randomized per account. Your us-east-1a might map to a different physical facility than another student's us-east-1a. AWS does this to spread load across AZs.



GCP Global Infrastructure

- GCP uses Regions and Zones (not "Availability Zones").
- A GCP **Region** is a specific geographic location (e.g., us-central1 is Iowa). A **Zone** is a single deployment area within a Region (e.g., us-central1-a). Each zone corresponds to one or more physical clusters of resources in a data center.
- Key difference from AWS: a GCP zone may exist within the same data center as another zone (separated logically, not necessarily physically in the same way AWS separates AZs). AWS AZs are explicitly in separate physical facilities.
- GCP also has **multi-regions** — larger geographic areas like US, EU, and ASIA — used primarily for storage replication (e.g., multi-region Cloud Storage buckets).

```
GCP INFRASTRUCTURE HIERARCHY
═══════════════════════════════════════════════

  Multi-Region: US
  ┌──────────────────────────────────────────┐
  │                                          │
  │  Region: us-central1 (Iowa)              │
  │   ┌─────────┐  ┌─────────┐  ┌─────────┐  │
  │   │Zone -a  │  │Zone -b  │  │Zone -c  │  │
  │   └─────────┘  └─────────┘  └─────────┘  │
  │                                          │
  │  Region: us-east1 (S. Carolina)          │
  │   ┌─────────┐  ┌─────────┐  ┌─────────┐  │
  │   │Zone -b  │  │Zone -c  │  │Zone -d  │  │
  │   └─────────┘  └─────────┘  └─────────┘  │
  │                                          │
  └──────────────────────────────────────────┘
```

Azure Global Infrastructure

- Azure uses Regions, Availability Zones, and a unique concept called **Geography**.
- An Azure **Geography** is a container of one or more Regions that preserves data residency and compliance boundaries (e.g., "United States" geography, "Europe" geography). This is important for organizations subject to regulations like GDPR.
- An Azure **Region** is a set of data centers deployed within a defined perimeter. Each Region may contain **Availability Zones** — physically separate locations within the Region with independent infrastructure. Not all Azure regions support AZs.
- Azure also has **Region Pairs** — each Region is paired with another Region in the same geography for disaster recovery. During planned maintenance, Azure updates one region in a pair at a time.

Cross-Platform Comparison — Global Infrastructure

Concept	AWS	GCP	Azure
Top geographic unit	Region	Multi-Region (storage) / Region	Geography / Region
Fault-isolation unit	Availability Zone (AZ)	Zone	Availability Zone
Naming example	`us-east-1a`	`us-central1-a`	Zone 1 in East US
Min AZs per region	3 (most regions)	3 (most regions)	3 (where supported)
Built-in region pairing	No (manual DR)	No (manual DR)	Yes (Region Pairs)
AZ ID randomization	Yes (per account)	No	No
Edge / PoP locations	400+ CloudFront PoPs	100+ Cloud CDN PoPs	190+ Azure CDN PoPs

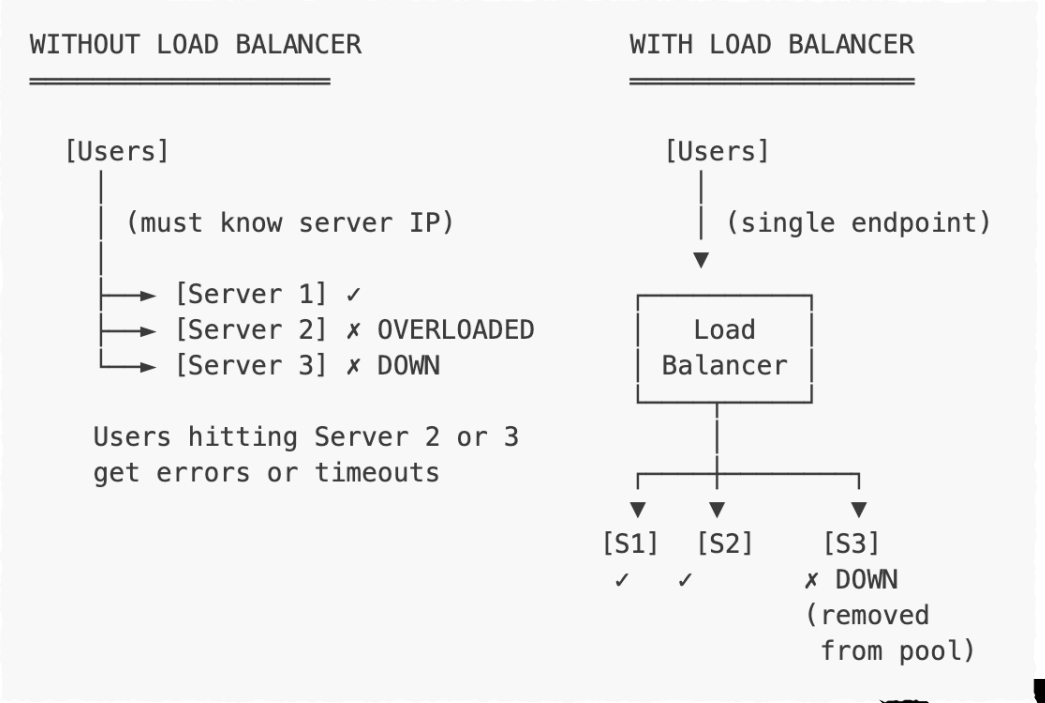
Designing for Fault Tolerance

- The reason this hierarchy exists is fault tolerance. The principle: **never rely on a single point of failure.**
  - **Single AZ deployment**: If that AZ goes down, your entire application is offline
  - **Multi-AZ deployment**: If one AZ fails, traffic shifts to instances in other AZs — application stays online
  - **Multi-Region deployment**: Protects against regional-level failures (rare, but catastrophic when they happen)
- All three cloud providers recommend deploying production workloads across at least two AZs. This is where load balancers become essential — they are the mechanism that distributes traffic across AZs and detects when an AZ is unhealthy.

Load Balancing Fundamentals

What Is a Load Balancer?

- A load balancer is a device (or cloud service) that sits between clients and a group of backend servers. It acts as a reverse proxy: clients connect to the load balancer's address, and the load balancer forwards each request to one of the backend servers based on a distribution algorithm.
- Without a load balancer, you would need to give users the IP address of a specific server. If that server goes down or becomes overloaded, users get errors. With a load balancer, the address never changes — the load balancer decides which healthy server should handle each request.



Why Use Load Balancers?

Load balancers provide three critical capabilities for cloud-native applications:

- **Increased capacity**: By distributing requests across multiple servers, a load balancer lets you handle more concurrent users than any single server could support. This is called **horizontal scaling** — adding more servers rather than making one server bigger (vertical scaling).
- **High availability**: The load balancer continuously monitors backend server health. If a server fails or becomes unresponsive, the load balancer stops sending it traffic and redistributes requests to healthy servers. Users experience no interruption.
- **Operational flexibility**: You can add or remove servers from the pool without downtime. This is essential for deployments — you can deploy a new version to a subset of servers while the load balancer keeps traffic flowing to the remaining healthy servers.

Layer 4 vs. Layer 7 Load Balancing

- Load balancers operate at different layers of the OSI networking model. The two that matter for cloud computing are Layer 4 (Transport) and Layer 7 (Application).
- **Layer 4 load balancers** make routing decisions based on network-level information: source/destination IP addresses and TCP/UDP port numbers. They do not inspect the actual content of the request. They are faster because they do less work per request, and they work with any TCP or UDP protocol.
- **Layer 7 load balancers** inspect the full application-layer content of HTTP/HTTPS requests. They can route based on the URL path, HTTP headers, cookies, query parameters, and even the request body. This enables sophisticated routing like sending /api/* requests to one group of servers and /static/* requests to another.

LAYER 4 vs LAYER 7 — WHAT THE LOAD BALANCER CAN SEE

```
LAYER 4 vs LAYER 7 — WHAT THE LOAD BALANCER CAN SEE
═══════════════════════════════════════════════════════

LAYER 4 (Transport)
  Sees: Source IP, Dest IP, Source Port, Dest Port, Protocol (TCP/UDP)
  Cannot see: URL, headers, cookies, request body
  Decision: "Send this TCP connection to Server 2"

  ┌──────────────────────────────────┐
  │ SRC: 203.0.113.50:54321          │
  │ DST: 10.0.1.100:443              │ ◄── L4 sees this
  │ Protocol: TCP                    │
  ├──────────────────────────────────┤
  │ GET /api/users HTTP/1.1          │
  │ Host: myapp.com                  │ ◄── L4 cannot see this
  │ Cookie: session=abc123           │
  └──────────────────────────────────┘

LAYER 7 (Application)
  Sees: Everything above PLUS URL path, headers, cookies, body
  Decision: "This is a request for /api/users — send to API server grou

  Can route based on:
    - URL path:    /api/*  → API servers,  /images/* → static servers
    - Host header: api.myapp.com → API,  www.myapp.com → web frontend
    - Cookie:      session=abc → sticky to Server 3
    - HTTP method:  GET vs POST can go to different backends
```

Load Balancing Algorithms

When a load balancer receives a request, it needs to decide which backend server should handle it. The most common algorithms are:

- **Round Robin**: Requests are distributed to servers in sequential order. Server 1, then Server 2, then Server 3, then back to Server 1. Simple and effective when all servers have equal capacity.
- **Weighted Round Robin**: Same as round robin, but servers are assigned weights. A server with weight 3 receives three times as many requests as a server with weight 1. Useful when servers have different hardware capabilities.
- **Least Connections**: New requests go to the server currently handling the fewest active connections. Better than round robin when request processing times vary widely (some requests are fast, some are slow).
- **Least Response Time**: New requests go to the server with the lowest average response time and fewest active connections. The most adaptive algorithm but requires the load balancer to track response latency.
- **IP Hash**: The client's IP address is hashed to determine which server receives the request. The same client always goes to the same server (a form of session persistence without cookies).

Cloud Load Balancer Offerings

AWS Elastic Load Balancing (ELB)

- AWS offers four load balancer types under the Elastic Load Balancing service. The Classic Load Balancer is the oldest and is effectively deprecated for new workloads — AWS recommends ALB or NLB instead.

Application Load Balancer (ALB) — Layer 7

- The ALB is the most commonly used load balancer for web applications. It operates at the HTTP/HTTPS layer and supports advanced routing features.
- Core concepts: A **listener** checks for incoming connections on a configured protocol and port (e.g., HTTPS on port 443). **Rules** on the listener evaluate conditions (path pattern, host header, HTTP method, etc.) and determine which **target group** receives the request. A **target group** is a collection of targets (EC2 instances, IP addresses, or Lambda functions) with a shared health check configuration.
- Key ALB features: host-based routing (route by domain name), path-based routing, HTTP/2 and WebSocket support, integration with AWS WAF (Web Application Firewall), native authentication via Cognito or OIDC, and support for Lambda functions as targets.

```
AWS APPLICATION LOAD BALANCER (ALB) ARCHITECTURE
═══════════════════════════════════════════════════════════

[Internet]
    │
    ▼
┌──────────────────────────────────────────────────────┐
│                ALB (Layer 7)                         │
│                                                      │
│  Listener: HTTPS :443                                │
│  ┌──────────────────────────────────────────────┐    │
│  │ Rule 1: IF path = /api/*                     │    │
│  │         THEN → Target Group: api-servers     │    │
│  │                                              │    │
│  │ Rule 2: IF path = /static/*                  │    │
│  │         THEN → Target Group: cdn-origin      │    │
│  │                                              │    │
│  │ Default: → Target Group: web-servers         │    │
│  └──────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────┘
        │              │              │
        ▼              ▼              ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│ TG: api      │ │ TG: cdn      │ │ TG: web      │
│ [EC2] [EC2]  │ │ [EC2]        │ │ [EC2] [EC2]  │
│ AZ-a  AZ-b   │ │ AZ-a         │ │ AZ-a  AZ-b   │
└──────────────┘ └──────────────┘ └──────────────┘
```

Network Load Balancer (NLB) — Layer 4

- The NLB operates at the TCP/UDP layer and is designed for extreme performance — it can handle millions of requests per second with ultra-low latency. Unlike the ALB, the NLB does not inspect HTTP content.
- Key NLB features: static IP address per AZ (important for whitelisting and DNS), source IP preservation (the backend server sees the client's real IP), TCP/UDP/TLS protocol support, and long-lived TCP connections (ideal for WebSocket, gRPC, and IoT workloads).

Gateway Load Balancer (GWLB) — Layer 3

- The GWLB is a specialized load balancer for deploying third-party network appliances (firewalls, intrusion detection systems, deep packet inspection).
- It operates transparently — traffic is routed through the appliance fleet without the application knowing.

Classic Load Balancer (CLB) — Layer 4/7 (Legacy)

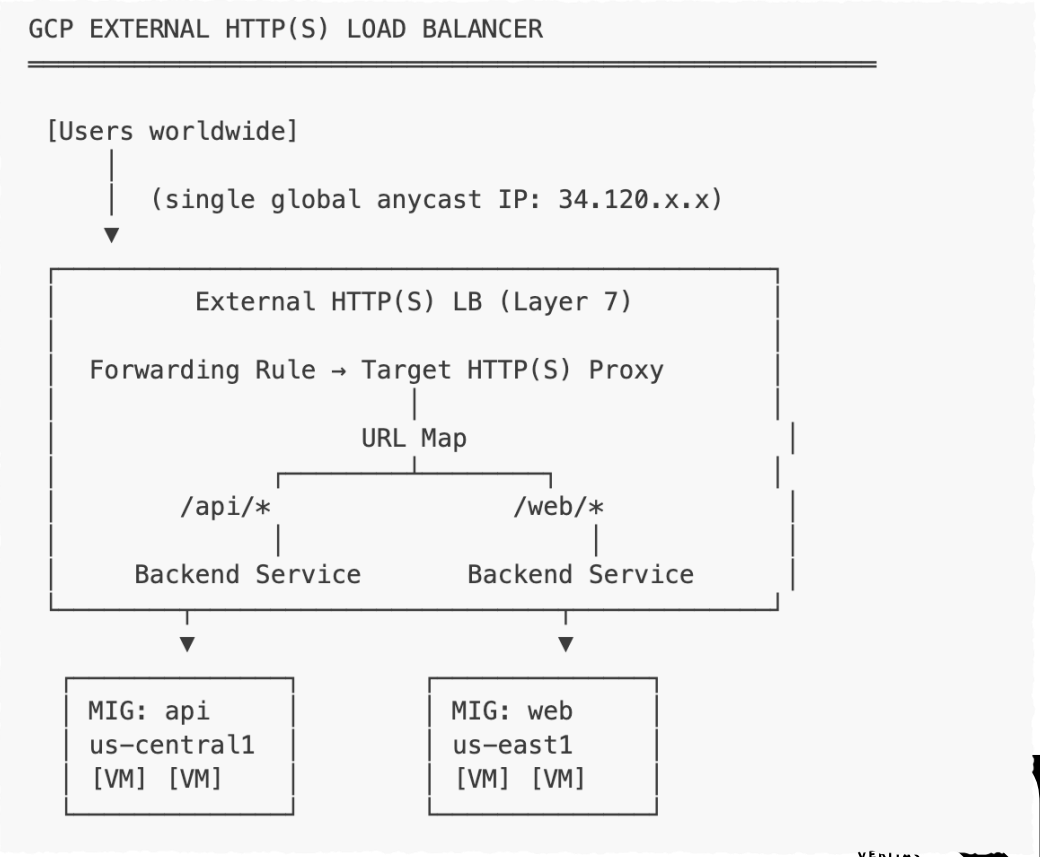
- The original AWS load balancer.
- It supports both Layer 4 and Layer 7 but lacks the advanced routing features of ALB and the performance characteristics of NLB.
- AWS recommends migrating existing CLBs to ALB or NLB.
- New applications should not use the Classic Load Balancer.

GCP Cloud Load Balancing

- GCP takes a fundamentally different approach from AWS: GCP load balancers are **global by default** for HTTP(S) traffic.
- A single GCP HTTP(S) load balancer can distribute traffic across backends in multiple regions using a single anycast IP address.
- AWS ALBs are regional — you need one ALB per region.

External HTTP(S) Load Balancer — Layer 7 (Global)

- This is GCP's primary load balancer for web applications. It uses a single global anycast IP, meaning DNS resolves to the same IP worldwide, and Google's network routes each user to the nearest healthy backend.
- Core concepts: A **forwarding rule** maps an external IP and port to a **target proxy**. The target proxy references a **URL map** that defines routing rules. The URL map routes to **backend services**, which contain one or more **instance groups** (managed or unmanaged) or **network endpoint groups (NEGs)**.



External TCP/UDP Network Load Balancer — Layer 4 (Regional)

- GCP's Layer 4 load balancer for non-HTTP traffic.
- Unlike the HTTP(S) LB, this one is regional (not global). Supports TCP and UDP.
- Preserves client source IP.

Internal HTTP(S) Load Balancer — Layer 7 (Regional)

- For internal microservice-to-microservice traffic within a VPC.
- Same URL map routing as the external version but only accessible from within the VPC.
- This is the GCP equivalent of an AWS internal ALB.

Internal TCP/UDP Load Balancer — Layer 4 (Regional)

- For internal Layer 4 traffic within a VPC.

Azure Load Balancing

Azure has multiple load balancing services, each designed for different scenarios.

- **Azure Load Balancer** — Layer 4 (Regional) - Azure's basic Layer 4 load balancer for TCP/UDP traffic. Available in two SKUs: Basic (free, limited features, being retired) and Standard (paid, production-ready). Supports internal and public configurations.
- **Azure Application Gateway** — Layer 7 (Regional) - Azure's Layer 7 load balancer, comparable to AWS ALB. Features include URL-based routing, SSL termination, Web Application Firewall (WAF) integration, cookie-based session affinity, and autoscaling.
- **Azure Front Door** — Layer 7 (Global) - Azure's global load balancer and CDN, comparable to GCP's global HTTP(S) load balancer combined with AWS CloudFront + ALB. It provides global HTTP routing, SSL offloading, WAF, and CDN caching in a single service.
- **Azure Traffic Manager** — DNS-based (Global) - Not a traditional load balancer — it works at the DNS level, directing users to the closest or healthiest regional endpoint. Similar to AWS Route 53 routing policies.

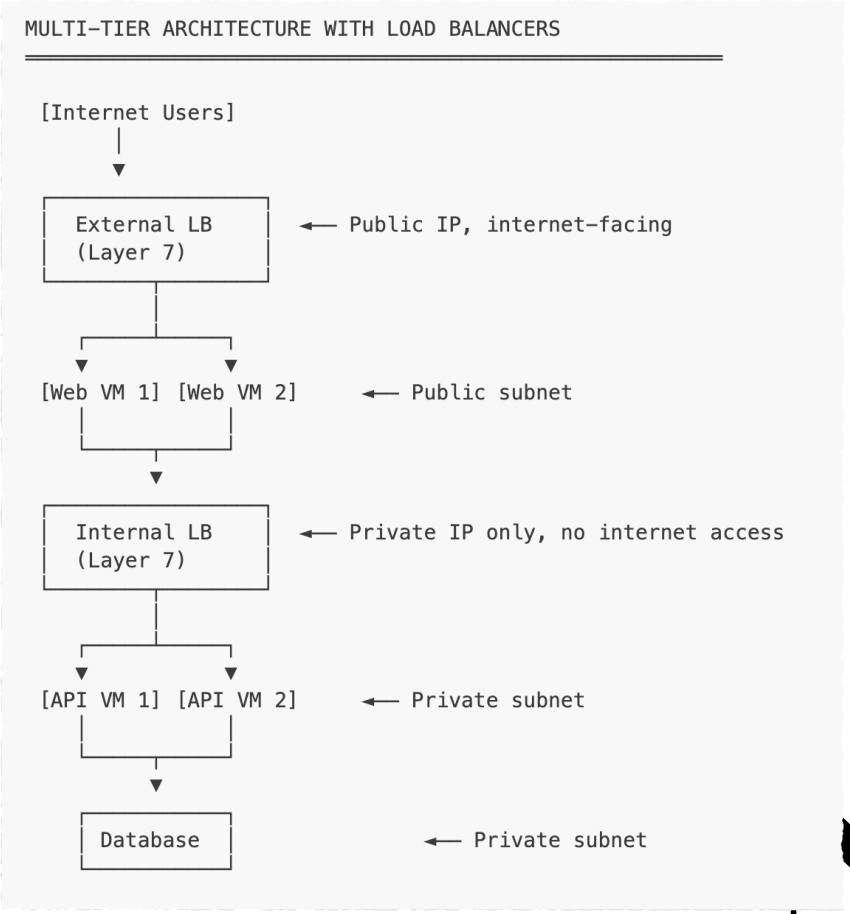
Cross-Platform Load Balancer Comparison

Feature	AWS ALB	AWS NLB	GCP HTTP(S) LB	GCP Network LB	Azure App Gateway	Azure Front Door
OSI Layer	7	4	7	4	7	7
Scope	Regional	Regional	**Global**	Regional	Regional	**Global**
Protocols	HTTP/S, gRPC	TCP, UDP, TLS	HTTP/S, gRPC	TCP, UDP	HTTP/S	HTTP/S
URL routing	Yes	No	Yes	No	Yes	Yes
Static IP	No	Yes	Yes (anycast)	Yes	No	N/A
WebSocket	Yes	Yes	Yes	N/A	Yes	Yes
WAF integration	AWS WAF	No	Cloud Armor	No	Built-in	Built-in
SSL termination	Yes	TLS passthrough	Yes	No	Yes	Yes
Source IP visible	Via header	Yes (native)	Via header	Yes (native)	Via header	Via header

Internet-Facing vs. Internal Load Balancers

All three providers support both configurations:

- **Internet-facing (external)**: Has a public IP address. DNS resolves publicly. Receives traffic from the internet and distributes it to backend servers that may be in private subnets.
- **Internal**: Has only a private IP address. Only accessible from within the VPC/VNet. Used for service-to-service communication in multi-tier architectures — for example, a web tier load balancer sends requests to an internal API tier load balancer.



Essential Load Balancer Features

Health Checks

- A health check is a periodic test the load balancer performs against each backend server to verify it can handle requests.
- If a server fails a configured number of consecutive health checks, the load balancer marks it as **unhealthy** and stops routing traffic to it.
- Once the server passes a configured number of checks again, it is marked **healthy** and traffic resumes.
- **Best practice:** Create a dedicated health check endpoint in your application (e.g., /healthz or /health) that verifies the application can connect to its database and other dependencies. Do not just return a 200 from a static route — that only proves the web server process is running, not that the application is actually functional.

Health check parameters across providers:

Parameter	AWS ALB/NLB	GCP HTTP(S) LB	Azure App Gateway
Protocol	HTTP, HTTPS, TCP	HTTP, HTTPS, TCP, gRPC	HTTP, HTTPS
Check interval	5–300 sec (default 30)	5–300 sec (default 5)	1–86400 sec (default 30)
Unhealthy threshold	2–10 (default 2)	1–10 (default 2)	1–20 (default 3)
Healthy threshold	2–10 (default 5)	1–10 (default 2)	1–20 (default 2)
Custom path	Yes (e.g., `/healthz`)	Yes	Yes
Expected response	200-399 (configurable)	200 (configurable)	200-399

SSL/TLS Termination (Offloading)

- SSL/TLS termination means the load balancer handles the HTTPS encryption and decryption instead of the backend servers. The client connects to the load balancer over HTTPS, and the load balancer connects to the backend servers over plain HTTP (within the private network).
- Why this matters: encrypting and decrypting HTTPS traffic consumes CPU. Offloading this work to the load balancer frees up backend server resources for application logic. It also centralizes certificate management — you install the TLS certificate on the load balancer once instead of on every backend server.
- AWS: ALB and CLB support SSL termination. NLB supports TLS termination or TLS passthrough.
- GCP: HTTP(S) Load Balancer terminates SSL. Can use Google-managed certificates (auto-renewed).
- Azure: Application Gateway and Front Door terminate SSL. Support for auto-managed certificates.

```
SSL/TLS TERMINATION
═══════════════════════════════════════

 [Client Browser]
        │
        │  HTTPS (encrypted, port 443)
        ▼
 ┌─────────────────┐
 │   Load Balancer │  ◄── TLS certificate installed here
 │   (terminates   │      Decrypts incoming HTTPS
 │    SSL/TLS)     │      Re-encrypts if needed (optional)
 └─────────────────┘
          │
          │  HTTP (unencrypted, port 80)
          │  (within private VPC — acceptable)
          ▼
 [Backend Servers]          ◄── No certificates needed
                                Less CPU overhead
```

Sticky Sessions (Session Affinity)

- Sticky sessions ensure that once a client is routed to a specific backend server, all subsequent requests from that client go to the same server for the duration of a session. This is implemented using cookies (Layer 7) or source IP hashing (Layer 4).
- When you need sticky sessions: your application stores session state in server memory (e.g., a shopping cart stored in a local variable). If requests bounce between servers, the session data is lost.
- When you should avoid sticky sessions: if your application uses a shared session store (Redis, database), sticky sessions are unnecessary and actually harmful — they create uneven load distribution because some servers end up with more long-running sessions than others.

Provider	Service	Mechanism
AWS	ALB	Application-generated cookie or LB-generated cookie (AWSALB)
AWS	NLB	Source IP / flow hash
GCP	HTTP(S) LB	Generated cookie (GCLB), header field, or HTTP cookie affinity
Azure	App Gateway	Cookie-based affinity (ARRAffinity)

Connection Draining (Deregistration Delay)

- When you remove a server from the load balancer pool (for deployment, scaling in, or maintenance), existing requests that are still being processed on that server need to finish. Connection draining gives in-flight requests a grace period to complete before the server is fully deregistered.
- Without connection draining, removing a server causes active requests to fail with connection reset errors. With connection draining, the load balancer stops sending new requests to the server but allows existing connections to complete within a configurable timeout (default 300 seconds on AWS).
- This is critical for zero-downtime deployments.

Provider	Feature Name	Default Timeout	Configurable
AWS	Deregistration delay	300 seconds	Yes (0–3600 sec)
GCP	Connection draining timeout	300 seconds	Yes (0–3600 sec)
Azure	Connection draining	30 seconds	Yes

Cross-Zone Load Balancing

- In a multi-AZ deployment, should the load balancer distribute traffic evenly across all backend instances regardless of AZ, or should it keep traffic within the same AZ?
- **Cross-zone load balancing enabled**: The load balancer distributes traffic evenly across all registered instances in all AZs. If AZ-a has 2 instances and AZ-b has 8 instances, each instance gets 10% of the traffic.
- **Cross-zone load balancing disabled**: Traffic is divided equally among AZs first, then distributed within each AZ. AZ-a's 2 instances each get 25% of traffic (overloaded). AZ-b's 8 instances each get 6.25% (underutilized).

Cross-Zone Load Balancing

- AWS: ALB has cross-zone enabled by default (and it cannot be disabled at the LB level). NLB has cross-zone disabled by default (can be enabled per target group).
- GCP: Cross-zone balancing is handled automatically by the global load balancer.
- Azure: Standard Load Balancer distributes across zones by default.

```
CROSS-ZONE LOAD BALANCING
═══════════════════════════════════════════

ENABLED (recommended):                DISABLED:
50% ÷ 10 instances = 10% each         50% to each AZ first

    AZ-a          AZ-b                    AZ-a          AZ-b

┌─────────┐   ┌─────────┐             ┌─────────┐   ┌─────────┐
│ [I1]10% │   │ [I3]10% │             │ [I1]25% │   │ [I3]6%  │
│ [I2]10% │   │ [I4]10% │             │ [I2]25% │   │ [I4]6%  │
│         │   │ [I5]10% │             │         │   │ [I5]6%  │
│         │   │ ...     │             │         │   │ ...     │
│         │   │ [I10]10 │             │         │   │[I10]6%  │
└─────────┘   └─────────┘             └─────────┘   └─────────┘

✓ Even distribution                   ✗ AZ-a overloaded
```

Auto Scaling

What Is Auto Scaling?

- Auto scaling automatically adjusts the number of compute instances in response to changing demand. When traffic increases, auto scaling launches new instances. When traffic decreases, it terminates excess instances. This ensures your application has enough capacity to handle load while minimizing cost during low-traffic periods.
- Auto scaling is tightly integrated with load balancing — new instances are automatically registered with the load balancer, and terminated instances are automatically deregistered.

```
AUTO SCALING OVER TIME
═══════════════════════════════════════════════

 Instances
    6│              ┌──────┐
    5│           /  │      │  \          ← Max capacity: 6
    4│        /     │      │    \
    3│     /        │      │      \
    2│──/───────────┴──────┴───────\──── ← Min capacity: 2
    1│                                   ← Desired: tracks demand
    0└──────────────────────────────────
      6am   9am  12pm  3pm  6pm  9pm

     └low┘ └peak──────┘ └low┘
      2 inst   scale out    scale in
               to 5-6       back to 2
```

Auto Scaling Components — AWS

AWS auto scaling centers on three components:

- **Launch Template** (replaces the older Launch Configuration): Defines what each new instance looks like — AMI ID, instance type, key pair, security groups, user data script, IAM role, storage configuration. Think of it as a blueprint for creating identical instances.
- **Auto Scaling Group (ASG)**: A collection of EC2 instances managed as a unit. You configure minimum size (never fewer than this), maximum size (never more than this), and desired capacity (the target count). The ASG maintains the desired capacity by launching or terminating instances.
- **Scaling Policies**: The rules that determine when and how to change the desired capacity.

Auto Scaling Components — GCP

- GCP uses **Managed Instance Groups (MIGs)** as the equivalent of AWS Auto Scaling Groups.
- **Instance Template**: The GCP equivalent of a Launch Template. Defines machine type, boot image, startup script, labels, network/subnetwork, and service account.
- **Managed Instance Group (MIG)**: A group of identical VM instances created from an instance template. MIGs support autoscaling, autohealing (automatic replacement of failed VMs), rolling updates, and load balancer integration.
- MIGs can be **zonal** (instances in a single zone) or **regional** (instances spread across multiple zones in a region — GCP manages the distribution automatically).

Auto Scaling Components — Azure

- Azure uses **Virtual Machine Scale Sets (VMSS)** as its auto scaling mechanism.
- **VMSS** is a group of identical VMs that can automatically increase or decrease in count based on demand or a schedule. Like AWS and GCP, you define a VM image, size, and configuration, and the scale set manages creating/destroying instances.
- Azure also provides **Azure Autoscale** as a platform-level service that can scale not just VMs but also App Services, Cloud Services, and other Azure resources.

Cross-Platform Auto Scaling Comparison

Concept	AWS	GCP	Azure
Instance blueprint	Launch Template	Instance Template	VMSS model (VM image + config)
Scaling group	Auto Scaling Group (ASG)	Managed Instance Group (MIG)	VM Scale Set (VMSS)
Multi-zone support	ASG spans multiple AZs	Regional MIG spans zones	VMSS spans AZs
Autohealing	ASG health checks	MIG autohealing policy	VMSS automatic repairs
Rolling updates	Instance refresh	Rolling update policy	Rolling upgrade policy
Min/max/desired	Yes	Yes (min replicas, max replicas)	Yes

Scale Out vs. Scale In

- **Scale out** means adding instances (increasing capacity). Triggers: increased demand, scheduled event, manual increase.
- **Scale in** means removing instances (decreasing capacity). Triggers: decreased demand, scheduled event, manual decrease.
- When scaling in, the auto scaling system must decide which instance to terminate. AWS uses a **termination policy** — the default policy first identifies the AZ with the most instances (to maintain balance), then within that AZ terminates the instance with the oldest launch configuration, or the one closest to the next billing hour.

Scaling Policy Types

- **Manual Scaling**: You directly change the desired capacity. Useful for known events (a scheduled product launch) or as a temporary override.
- **Scheduled Scaling**: You define a schedule — for example, scale to 10 instances every weekday at 8am, scale back to 2 instances at 8pm. Useful when traffic patterns are predictable and recurring.
- **Dynamic Scaling (Reactive)**: The system monitors a metric (CPU utilization, request count, custom metric) and adjusts capacity when the metric crosses a threshold.
- Types of dynamic scaling on AWS:
  - **Target tracking**: You set a target value (e.g., "keep average CPU at 50%"). The ASG automatically adjusts to maintain that target. Simplest and most commonly used.
  - **Step scaling**: Define step adjustments — if CPU is 50–70%, add 1 instance; if CPU is 70–90%, add 3 instances. More granular control than target tracking.
  - **Simple scaling**: Add or remove a fixed number of instances when a CloudWatch alarm fires. Requires a cooldown period before the next action. Older approach, less recommended.
- **Predictive Scaling**: Uses machine learning to forecast traffic patterns based on historical data and pre-provisions capacity before demand arrives. Available on AWS and Azure. GCP offers a similar concept through scheduled scaling combined with monitoring.

Cooldown Periods and Warm-Up Time

- **Cooldown period** (AWS simple scaling): After a scaling activity completes, the ASG waits for a cooldown period (default 300 seconds) before responding to additional alarms. This prevents thrashing — rapidly scaling out and in before new instances have had time to absorb load.
- **Instance warm-up time**: When a new instance launches, it takes time to boot, run user data scripts, and become fully operational. During warm-up, the instance should not be counted toward scaling metric calculations — otherwise the system might think it needs even more instances because the new one hasn't started handling traffic yet.

Concept	AWS	GCP	Azure
Cooldown	Configurable per policy (default 300s)	Stabilization period (60s default)	Cooldown period (5 min default)
Warm-up	Estimated warm-up time per policy	Initialization period (configurable)	Configurable in autoscale rules

Putting It All Together

How Load Balancing and Auto Scaling Work Together

How Load Balancing and Auto Scaling Work Together

In a production deployment, a load balancer and an auto scaling group are always used together:

- The auto scaling group maintains a fleet of instances across multiple AZs
- The load balancer distributes incoming traffic across the healthy instances
- When demand increases, the auto scaling group launches new instances and registers them with the load balancer
- When demand decreases, the auto scaling group deregisters instances from the load balancer (with connection draining) and terminates them
- If an instance fails a health check, both the load balancer (stops sending traffic) and the auto scaling group (terminates and replaces it) respond

```
COMPLETE ARCHITECTURE
═══════════════════════════════════════════════════

                [DNS: myapp.com]
                       │
                       ▼
              ┌─────────────────┐
              │  Load Balancer  │
              │  (ALB / GCP     │
              │   HTTP(S) LB)   │
              └─────────────────┘
                       │
        ┌──────────────┼──────────────┐
        ▼              ▼              ▼
     [AZ-a]         [AZ-b]         [AZ-c]

    ┌──────┐       ┌──────┐       ┌──────┐
    │[VM-1]│       │[VM-3]│       │[VM-5]│
    │[VM-2]│       │[VM-4]│       │      │
    └──────┘       └──────┘       └──────┘
        └──────────────┴──────────────┘
              Auto Scaling Group
              Min: 2 | Max: 10 | Desired: 5

      ┌──────────────────────────┐
      │ CloudWatch / Monitoring  │
      │ CPU > 70% → scale out    │
      │ CPU < 30% → scale in     │
      └──────────────────────────┘
```

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS